How-To Library

Informatica

Best Practices for Optimizing Performance in PowerExchange for Netezza

© Copyright Informatica LLC 2016, 2022. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html

Abstract

This article describes general reference guidelines and best practices to help you tune the performance of PowerExchange for Netezza. You can tune the key hardware, driver, Netezza database, Informatica mapping, and session parameters to optimize the performance of PowerExchange for Netezza. This article also provides information on how to avoid common errors when you use PowerExchange for Netezza.

Supported Versions

- PowerExchange for Netezza 10.2 or later

Table of Contents

Overview of PowerExchange for Netezza. . . . 2
Performance Tuning Areas. . . . 3
Tune the Hardware. . . . 3
CPU Frequency. . . . 3
NIC Card Ring Buffer Size. . . . 4
Tune the Netezza Database Parameters. . . . 5
Tune the Driver Parameters. . . . 6
Tune the Mapping. . . . 7
Ports Precision. . . . 7
Data Movement Mode. . . . 7
Data Type Mapping. . . . 7
Tune the Session. . . . 7
Netezza Source Sessions. . . . 8
Netezza Target Sessions. . . . 9
General Guidelines for Netezza Target Sessions. . . . 16
Session Property Recommendations for ODBC Settings. . . . 18
Pre-SQL and Post-SQL Commands. . . . 20
Stored Procedure Calls. . . . 21
Avoiding Common Errors with PowerExchange for Netezza 21
Alternatives to Partitioning. . . . 21
Serialization Errors. . . . 21
Serializable Transaction Isolation. . . . 24
Unavailability of Locks on Netezza Tables. . . . 25
Buffer Size. . . . 27

Overview of PowerExchange for Netezza

You can connect to the Netezza Performance Server from PowerCenter to read data from and load data to Netezza tables. You can use either the PowerExchange for Netezza connection or the default ODBC connection to connect to Netezza.

If you use the ODBC connection, you must configure the Netezza ODBC driver on the machine where the PowerCenter Integration Service process runs. The Netezza Performance Server integrates database, server, and storage in a single system. The PowerCenter Integration Service extracts data from or loads data to Netezza tables through external tables. The PowerCenter Integration Service uses the bulk load utility on the external table to extract and load data.

Performance Tuning Areas

Performance tuning is an iterative process in which you analyze the performance, use guidelines to estimate and define parameters that impact the performance, and monitor and adjust the results as required.

You can optimize the performance of PowerExchange for Netezza mappings by tuning the following areas:

- Hardware
- Database
- Driver
- Mapping
- Session

Note: The performance testing results listed in this article are based on observations in an internal Informatica environment using data from real-world scenarios. The performance of PowerExchange for Netezza might vary based on individual environments and other parameters even when you use the same data.

Tune the Hardware

You can tune the following hardware parameters to optimize the performance of the machine where the PowerCenter Integration Service runs:

- CPU frequency
- NIC card ring buffer size

CPU Frequency

Dynamic frequency scaling adjusts the frequency of the processor on-the-fly either for power savings or to reduce heat. Ensure that the CPU operates at least at the base frequency.

When CPUs are underclocked, where they run below the base frequency, the performance degrades by 30% to 40%. Informatica recommends that you work with your IT system administrator to ensure that all the nodes on the cluster are configured to run at their supported base frequency.

To tune the CPU frequency for Intel multicore processors, perform the following steps:

1. Run the lscpu command to determine the current CPU frequency, base CPU frequency, and the maximum CPU frequency that the processor supports.
2. Request your system administrator to perform the following tasks:
    a. Increase the CPU frequency to the supported base frequency.
    b. Change the power management setting to **OS Control** at the BIOS level.
3. Run CPU-intensive tests to monitor the CPU frequency in real time and adjust the frequency for improved performance. On Red Hat operating systems, you can install a monitoring tool such as cpupower.
4. Work with your IT department to ensure that the CPU frequency and power management settings are persisted even for future system restarts.

NIC Card Ring Buffer Size

NIC configuration is a key factor in network performance tuning. When you deal with large volumes of data, it is crucial that you tune the Receive (RX) and Transmit (TX) ring buffer size. The ring buffers contain descriptors or pointers to the socket kernel buffers that hold the packet data.

You can run the ethtool command to determine the current configuration. For example, run the following command:

```
# ethtool -g eth0
```

The following sections show a sample output:

```
Ring parameters for eth0:
Pre-set maximums:
RX: 2040
RX Mini: 0
RX Jumbo: 8160
TX: 255

Current hardware settings:
RX: 255
RX Mini: 0
RX Jumbo: 0
TX: 255
```

The **Pre-set maximums** section shows the maximum values that you can set for each parameter. The **Current hardware settings** section shows the current configuration details.

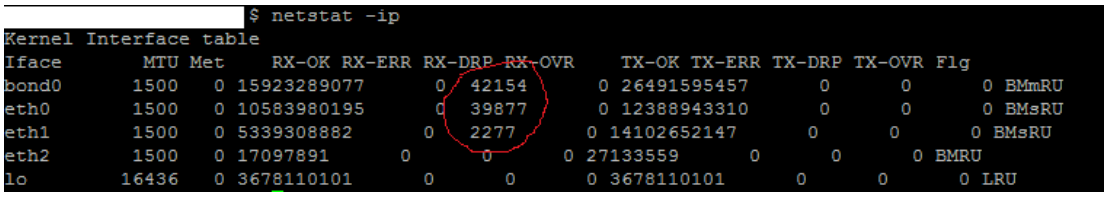
A low buffer size leads to low latency. However, low latency comes at the cost of throughput. For greater throughputs, you must configure large buffer ring sizes for RX and TX.

Informatica recommends that you use the ethtool command to determine the current hardware settings and the maximum supported values. Then, set the values based on the maximum values that are supported for each operating system. For example, if the maximum supported value for RX is 2040, you can use the ethtool command as follows to set the RX value to 2040:

```
# ethtool -G eth0 RX 2040
```

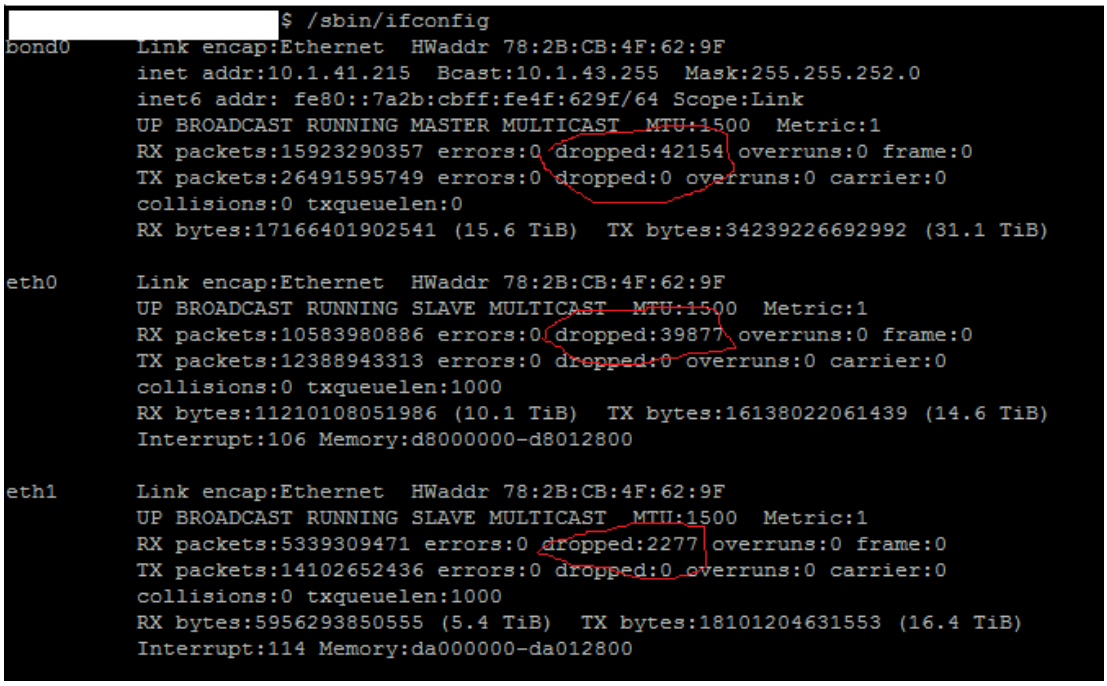
If you set a low ring buffer size for data transfer, packets might get dropped. To find out if packets were dropped, you can use the netstat and ifconfig commands.

The following image shows a sample output of the netstat command:



The RX-DRP column indicates the number of packets that were dropped. Set the RX value such that no packets get dropped and the RX-DRP column shows the values as 0. You might need to test several values to optimize the performance.

The following image shows a sample output of the ifconfig command:



The status messages indicate the number of packets that were dropped.

Tune the Netezza Database Parameters

You can tune the Netezza database parameters to optimize the performance of leveraging data effectively using PowerExchange for Netezza.

Consider the following best practices when you configure the Netezza database:

- Choose the right distribution key for Netezza tables to distribute the data efficiently. A bad choice of the distribution key might result in performance degradation. Use an integer for the column ID that gets incremented in a sequence, so that data in Netezza is distributed evenly.
- Do not exceed the limit of 31 concurrent write transactions for a Netezzza instance on a server. If you exceed 31 concurrent load processes, the loads queue up until the other sessions complete. You need to maintain the concurrent load processes below 31 to prevent interference with other processes that are trying to load data into Netezza.
- Use NZSQL Generate Statistics to update the statistics for large tables to optimize performance.
- Use the Zone Maps information as Zone Maps are critical for SQL read performance.
- To load many records to a Netezza table, suspend the materialized view ALTER VIEWS ON MATERIALIZE SUSPEND.
- If there are many logically deleted records or when the NZload fails to complete, perform one of the following NZReclaim operations:
  - To perform a block-level reclamation, which is optimal for failed loads or records that are in the same distribution range, run the following command from the database:

    ```
    nzreclaim -blocks -u user -pw password -host alpha -db emp
    ```
  - To perform a record-level reclamation, run the following command from the database:

    ```
    nzreclaim -records -u user -pw password -host alpha -db emp
    ```

For information about performance tuning for Netezza databases, see the following website:
http://nztips.com/category/performance-tuning-netezza/

Tune the Driver Parameters

If you use the ODBC connection, use only the certified ODBC driver version with Netezza for optimal performance.

Consider the following recommendations when you configure the ODBC driver:

DebugLogging

A Boolean property that activates debug logs. To enable logging, select the property in the Windows dialog box. On UNIX, set the Boolean value as 1 or True. Default is disabled. Informatica recommends that you enable this flag during regular production operations to avoid performance degradation. Use the DebugLogging option only for debugging.

Prefetch Count

A numeric value that sets the number of rows the driver fetches at a time from a Netezza database. Default is 256 rows.

To tune the performance of an application, set a value that optimizes network use versus memory use. The higher the value you set, the more memory is required to hold these rows.

Fetching multiple rows might result in the following error: `Row error occurred while fetching data from database`. The probability increases with a higher Prefetch Count value. To avoid this error, set `OptimizeODBCRead` option value as `NO` in the custom properties when you configure the Informatica domain. With this setting, the PowerCenter Integration Service fetches a single row instead of multiple rows.

Socket Buffer Size

A numeric property that specifies the size of the communication buffer in bytes. The range is 1 K to 32 K. Default is 8 K.

The socket buffer size is the number of bytes, for each network packet, that is transferred between the database server and clients. When set correctly, this attribute optimizes performance.

Character Translation Option

The Netezza system uses the Latin9 character encoding for char and varchar types. The character encoding for many Windows systems is similar, but not identical. If the database includes characters that use only the basic subset of letters (a-z or A-Z), numbers (0-9), or punctuation characters, select the Optimize for ASCII character set option for the Windows driver to enhance the performance. However, if you use characters such as the Euro symbol or other characters that are outside the basic set, do not select the Optimize option. The configuration converts the entered characters to the proper encodings so that they appear correctly in the query result.

UnicodeTranslationOption

For UNIX or Linux drivers, UnicodeTranslationOption specifies the Unicode encoding value. Valid values are UTF-8, UTF-16, and UTF-32. In the UnicodeTranslationOption for UNIX clients, a value other than UTF-8 affects or degrades the performance. Informatica recommends not to change this option.

Security Level

The level of security for the connection. A secured ODBC connection is slower than unsecured. Therefore, set this value to preferredUnSecured if the driver performance is of higher priority than security.

Tune the Mapping

You can tune the following parameters at the mapping level to achieve optimal performance:

- Ports precision
- Data movement mode
- Data type mapping

For more information, see the *Informatica Performance Tuning Guide*.

Ports Precision

Precision is the maximum number of significant digits for numeric data types, or maximum number of characters for string data types. For numeric data types, precision includes scale.

You can tune the precision in PowerCenter repository mappings.

When mappings contain ports with a larger precision than required, the mapping performance degrades. Informatica recommends that you set the precision judiciously for all source ports, transformation ports, and target ports. For instance, if a string port can handle data of a maximum of 200 characters, set the precision to 200. Do not set the precision to a high value such as 1000.

Data Movement Mode

The data movement mode specifies the mode that the PowerCenter Integration Service must use while moving data.

The data movement mode affects how the PowerCenter Integration Service enforces code page relationships and code page validation. It can also affect performance. Applications can process single-byte characters faster than multibyte characters.

You can tune the data movement mode in PowerCenter repository mappings.

When you create a PowerCenter Integration Service, you can specify the mode based on the type of data you want to move, single byte or multibyte data. For example, if the data does not contain any UTF-8 data, specify the data movement mode as ASCII.

Data Type Mapping

When the PowerCenter Integration Service reads source data, it converts the native data types to the comparable transformation data types before transforming the data. When the PowerCenter Integration Service writes data to a target, it converts the transformation data types to the comparable native data types.

When you map source ports to transformation ports and then to target ports, avoid unnecessary data type conversions. For instance, do not map a port of the string data type to a port of the date data type. Ensure that you map ports to the same data type in all components of the mapping. Also, remove all unconnected ports from the mapping.

Tune the Session

You can tune the session properties to achieve optimal performance when you extract data from or load data to Netezza.

Netezza Source Sessions

You can tune the following session parameters for Netezza sources to extract data from Netezza:

- Partitioning
- Session on grid
- Pipeline
- Pushdown optimization

You can also follow some general guidelines when you configure Netezza source sessions.

For more information, see the *Informatica Performance Tuning Guide*.

Partitioning

You can use partitioning to increase the number of transformation threads and to enhance session performance. Netezza internally divides the data of each table into multiple data slices based on a distribution key. Informatica recommends you to use this feature to enhance the performance by specifying a different source qualifier predicate on each partition based on the distribution key such that the entire data is distributed as uniformly as possible among all the partitions.

For example, in a table, the distribution key falls in the range 1 to 100, and the data is uniformly divided among four buckets of the ranges 1-25, 26-50, 51-75, and 76-100. In this scenario, the approach is to create four partitions, each containing data from the mentioned ranges.

When you configure partitioning for a session, you must adhere to the following guidelines:

- Set the partitioning type to pass-through for Netezza targets.
- Do not enter different column names for the source filter across partitions. Specify different values for the same column.
- Do not enter different values for the user-defined join across partitions.

Session on Grid

The PowerCenter Integration Service distributes workflows and session threads to the nodes on a grid to optimize performance and scalability. Informatica recommends to use this feature when more than one PowerCenter Integration Service is available for running a session. Ensure that you install the Netezza ODBC driver and PowerExchange for Netezza Service components on each of the PowerCenter Integration Service nodes that participate in a grid.

Pipeline

You can run multiple pipelines in a session. One pipeline represents one data flow. You can run the pipelines in any order.

You can create multiple pipelines to extract data from either a single table or multiple source tables because you can run concurrent Select queries on a single table. Informatica recommends to use this option when you load one source data into multiple target tables.

Pushdown Optimization

To enhance the performance, you can push transformation logic to the source database when the Source Qualifier transformation contains an SQL override. You cannot configure pushdown optimization when you use PowerExchange for Netezza.

Use pushdown optimization when you use the ODBC connection to push the transformation logic to Netezza. If the source and target databases are the same, you can configure full pushdown for improved performance.

Pushdown optimization forces the SQL to run on the Netezza server, and does not require data to move back and forth over ODBC, therefore enhancing the performance. The amount of transformation logic that the PowerCenter Integration Service pushes to the source database depends on the database, the transformation logic, and the mapping configuration. The PowerCenter Integration Service processes all transformation logic that it cannot push to a database.

When you push transformation logic to the database, ensure that the database has enough resources to process the queries faster. Otherwise, there could be a performance degradation.

When you use pushdown optimization, you must emphasize on processing mappings sequentially for pushdown instead of concurrent processing. If you initially design the mappings for PowerExchange and then decide to adopt pushdown optimization, you must redesign all the mappings to run sequentially. Informatica recommends that you must decide whether you want to use pushdown optimization in the initial stage.

General Guidelines for Netezza Source Sessions

Consider the following best practices when you configure the source properties for a session that reads data from Netezza:

- In the session properties, avoid using the EscapeCharacter option to improve the performance as escape characters require additional parsing of the source data.
- When the data contains NCHAR and NVARCHAR columns, set the data movement mode for the PowerCenter Integration Service to Unicode.
- When you configure an SQL override query, enclose the table names and column names within double quotes.
- When you configure a user defined join and if two fields have the same name in both the tables, the session fails. You must use an SQL override with aliases for ambiguous column names.
- The metadata of the source tables in the Netezza mappings must match the metadata in the Netezza database. If you make changes to the data in the database after you create the mappings, the session fails with the following error message:
  ```
  [ERROR] The PowerCenter Integration Service fails the session, as Netezza might not be able to serialize execution of queries
  ```

Netezza Target Sessions

You can tune the following session parameters for Netezza targets when you load data to Netezza:

- Partitioning
- Session on grid
- Pipeline
- Ignore key constraint
- Update strategy

You can also follow some general guidelines for bulk load, single data row inserts, and concurrent workflow when you configure Netezza target sessions.

For more information, see the *Informatica PowerCenter Performance Tuning Guide*.

Partitioning

Use partitioning when you want to increase the number of transformation threads and session performance. Partitioning is an add-on feature of PowerCenter that you can buy at an extra cost.

You can map one column of application source qualifier for a uniform distribution of records to the distribution column of the target table.

Consider the following points when you create partitions for a session:

- Configure partitioning only for a large data load or for complex transformations.
- By default, the PowerCenter Integration Service leverages the distribution key information in Netezza based on the datasliceid function for parallel processing. You can use a custom filter provided that you know the location of the data.
- The data in the tables in Netezza must be evenly distributed for better performance.
- For bulk mode, set the partitioning type to pass-through for Netezza targets.
- For normal mode, you can set to database, hash, key range, pass-through, or round-robin partitioning.
- Do not enter an SQL override query for a partition.
- Do not delete or update from more than one partition within a session. Verify that you have enabled the Delete and Update properties on the **Mapping** tab for only one partition.
- To synchronize each partition within a session, you can configure the insert, delete, update, ignore key constraint, or duplicate row handling options.

You can use partitioning to create multiple partitions. The throughput gain with increase in partitions might not always be linear. Partitioning is CPU bound. Therefore, you must configure partitioning based on the available hardware in your environment.

Session On Grid

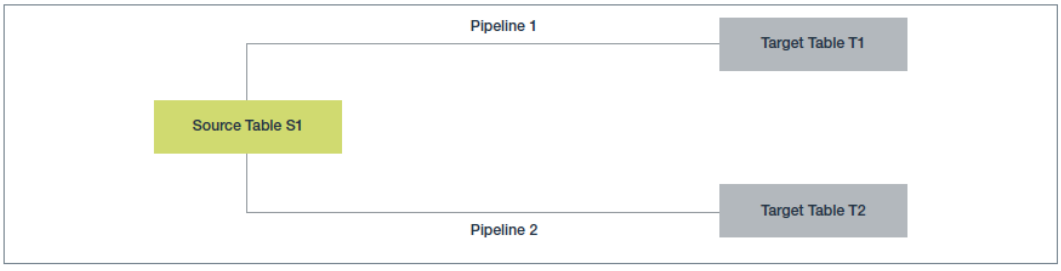
The PowerCenter Integration Service distributes workflows and session threads to the nodes in a grid to increase performance and scalability. You can use this feature if more than one PowerCenter Integration Service is available to run the session. Install the Netezza ODBC driver and PowerExchange for Netezza Service components on each of the PowerCenter Integration Service nodes that participate in the grid.

Configuring Multiple Pipelines

You can run multiple pipelines within a session. One pipeline represents one data flow. Netezza does not enforce primary and foreign key constraints on the table and there is no parent-child relationship between the tables in Netezza. You can run all pipelines in a workflow in any order because the order of execution does not affect the tables in Netezza.

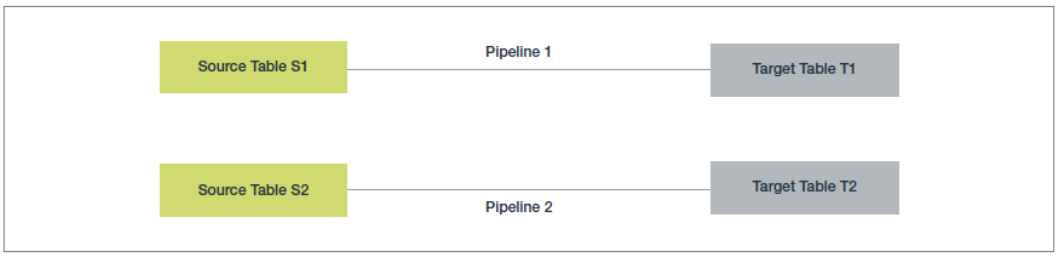
Consider the following two scenarios in which you can create multiple pipelines for loading data into Netezza:

- Each pipeline is associated with a unique target table.
  The following image shows pipelines 1 and 2 that loads or updates data to target tables T1 and T2:



  In this scenario, where the target tables are different, both pipelines can perform any operation, insert, update, or delete, on their respective target tables.
- Each pipeline is associated with multiple instances of the same target table.

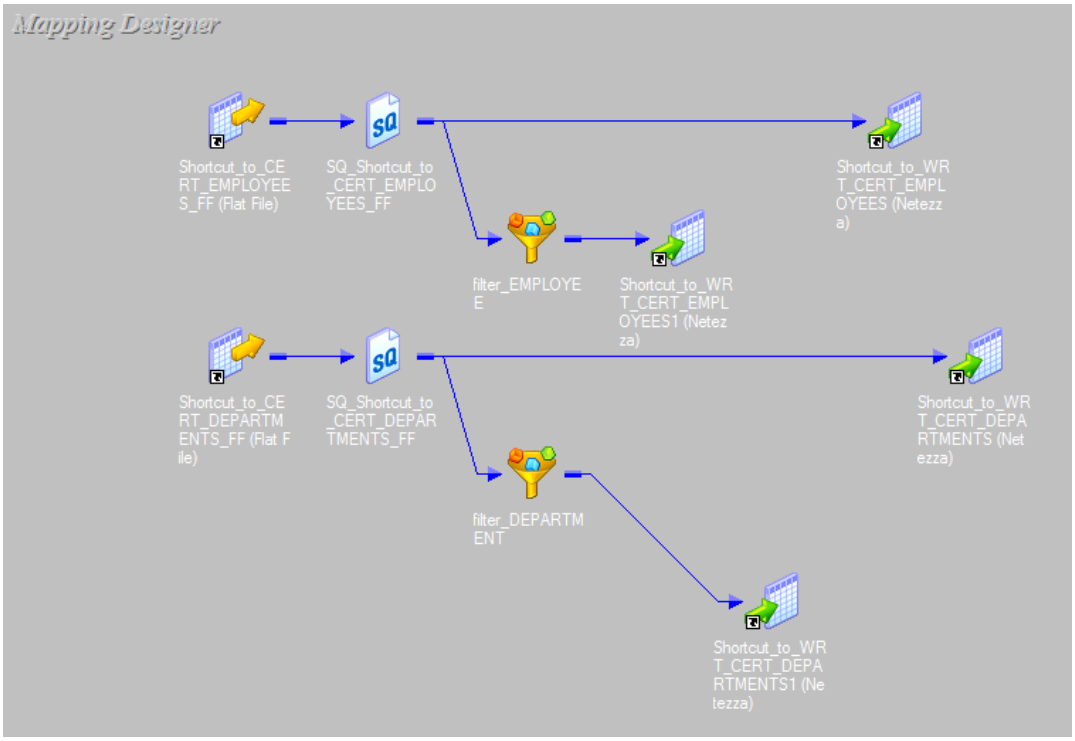
The following image shows two pipelines 1 and 2 that simultaneously loads or updates a single target table:



In this load or update scenario for multiple pipelines, consider the following best practices:

- Configure all pipelines to insert data into the target table because Netezza allows parallel inserts in a table. Netezza does not allow simultaneous execution of any other operation, update or delete, with insert.
- Do not configure pipelines for a single target such that multiple update, delete, or update and delete occur in parallel.

The following image shows a classic example of a scenario with more than one pipeline:



Because pipeline is not dependent with the partitioning feature, you can configure a pipeline with or without partitioning.

Ignore Key Constraint

When you enable the Ignore Key Constraint option, you can load duplicates into Netezza. To load unique data into Netezza, do not enable this option. By default, this option is disabled. The performance of the connector improves when you enable this option.

If you want to read from and load to Netezza, Informatica recommends you to enable the Distinct and Ignore Key Constraint flags, which manages duplicates at the source and enhances the performance of the connector. You must try to eliminate duplicates at the source so that there is no overhead on PowerExchange to remove duplicates.

Update Strategy

When you configure the session, consider the key constraints along with the duplicate row handling option for an effective update strategy.

You can set the insert, update as update, update else insert, update as insert, or delete options for the target table. The performance of the update else insert option is considerably low. Informatica recommends to use the update as insert option instead of the update else insert option.

For any of the update strategies for the target table, configure the source table strategy. When reading source data, the PowerCenter Integration Service marks each row with an indicator to specify which operation to perform when the row reaches the target. The source table indicator can have different settings according to the update strategy for the target table. You can set this value on the **Task** tab by selecting one of the Treat source rows as options, such as insert, update, delete, or data driven.

The following table describes the values of Treat Source Rows As Options:

Treat Source Row As Options	Description	Recommendation
Insert	Marks all rows to insert into the target.	Turn on the Insert flag in the target table property.
Delete	Marks all rows to delete from the target.	Turn on the Delete flag in the target table property.
Update	Marks all rows to update into the target. You can further define the update type in the target.	Turn off the Insert and Delete flags in the target, and select any type of update in the target table.
Data Driven	The PowerCenter Integration Service uses Update Strategy transformations in the mapping to determine the operation on a row-by-row basis. You can define the update operation in the target options.	If the mapping contains an Update Strategy transformation, the default option is Data Driven. You can also use this option when the mapping contains Custom transformations configured to set the update strategy.

Example of Update Else Insert and Update As Insert Strategy

Consider a scenario where two databases have tables with identical schema that contains the following information for each employee:

- Source Table: - S (DID int, EID int, Hours int) with data `111,101,2; 111,202,108; 111,101,22; 111,303,34; 111,404,45; 111,101,32;`
- Target Table: - T (DID int, EID int, Hours) with data `111, 101, 66; 111, 505, 5;`

For both source and target tables, S and T, consider DID and EID as the primary keys and the Duplicate Row Handling option as `FIRST`.

The objective is to update the target table, T, using the source table, S. You can update either by using the UpdateAsInsert or the UpdateElseInsert option. The end results for both update operations are identical, but the performance differs.

The following sections describe the performance difference between the two update strategies:

UpdateElseInsert

When you configure this update strategy, the PowerCenter Integration Service runs the SQL update command, followed by the Insert operation on the target table, T.

The PowerCenter Integration Service performs the following tasks:

1. The PowerCenter Integration Service executes an update of rows that exist in both the target and source: The following table displays the data in target table T that results after the PowerCenter Integration Service runs the SQL statement:

DID	EID	Hours	Comments
111	101	2	Key (111,101) found in the target, therefore the Hours column is updated with the value 2. Value is taken from the first row of source table, 111, 101, 2).
111	505	5	No change in target as key (111, 505) does not match.

2. The PowerCenter Integration Service runs an insert of rows that exist in the source but not in the target. The following table shows the results of the operation:

DID	EID	Hours	Comments
111	101	2	No Change
111	202	108	Key(111,202) not found in target so the source row 111,202,108 inserted
111	303	34	Key(111,303) not found in target so the source row 111,303,34 inserted
111	404	45	Key(111,404) not found in target so the source row 111,404,45 inserted
111	505	5	No Change

UpdateAsInsert

For this update strategy, the PowerCenter Integration Service runs a delete of all rows that exist in both the source and target tables. The PowerCenter Integration Service then runs an insert of all rows from the source to the target only after taking the first value where duplicates exist in the source.

The following table T displays the data that results after the PowerCenter Integration Service runs the SQL statement:

DID	EID	Hours	Comments
111	101	2	Inserted
111	202	108	Inserted
111	303	34	Inserted
111	404	45	Inserted
111	505	5	No change

The end result of both the UpdateAsInsert and UpdateElseInsert operations is exactly the same, although the PowerCenter Integration Service runs different SQL commands. For UpdateAsInsert, the PowerCenter Integration Service runs two commands to complete the operation. For UpdateElseInsert, the PowerCenter Integration Service runs only one command. For an update operation, Netezza does not perform an update but performs a delete followed by an insert operation. The UpdateAsInsert process is more efficient, as it

performs a delete of all rows followed by an insert of all rows. Informatica recommends to use the UpdateAsInsert strategy for better performance.

Mapping with Update Else Insert on Dimension Tables

To perform an Update Else Insert operation on the dimension table, design the mapping to load data in two steps:

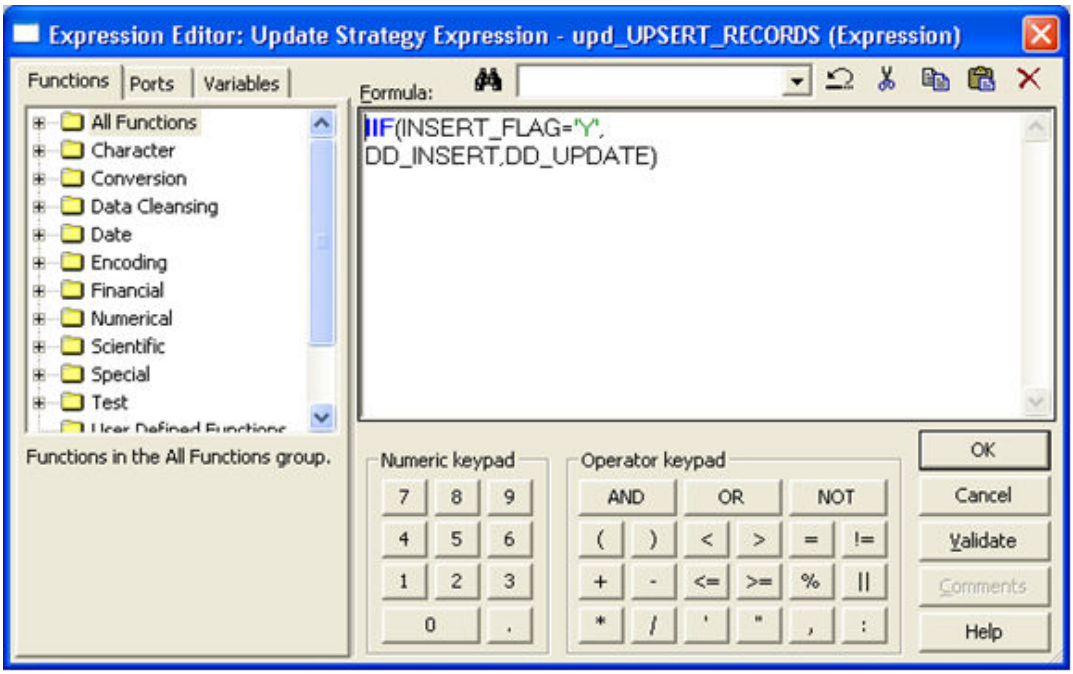
1. In the target table, update all the rows whose keys are present in the source table.
2. In the target table, insert all the rows from the source table whose keys are not present in the target table.

You can redesign the mapping by using the Router transformation for better performance. Break down the UpdateElseInsert mapping to a two-pipeline mapping using a Router transformation. Based on the lookup, one pipeline inserts the new record to the final dimension table, and the other pipeline inserts the updated records into an intermediate staging table. Next, run a post-SQL query for the session, which updates the final dimension table with only the records from the staging table. The operation results in performing two large inserts, and then a comparatively smaller single update statement, which boosts the performance.

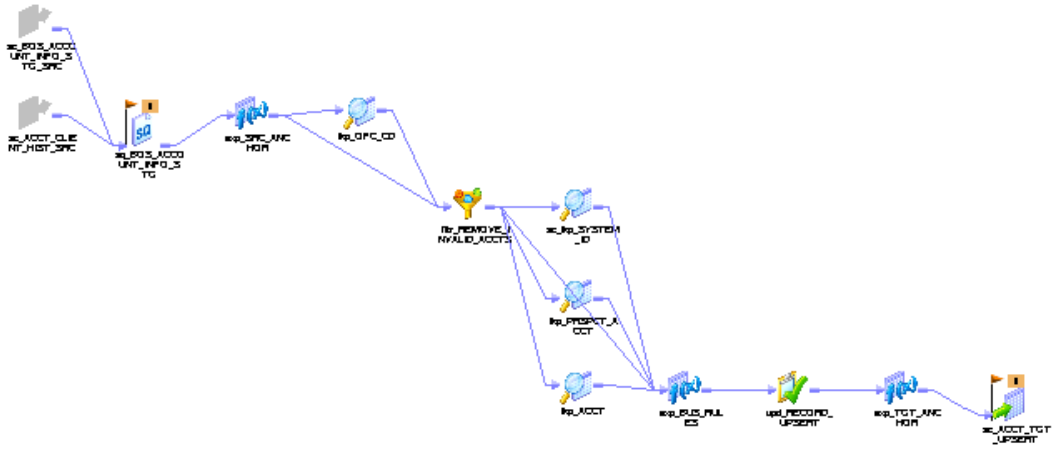
Best Practices to Avoid Serialization Errors with Data Upload

Consider the following guidelines to avoid a serialization error:

- When you configure an upsert operation in a mapping in PowerCenter to upsert data in a Netezza target, you can use a single flow to load data into the same target table to avoid a serialization error.
- To configure a mapping to include an insert and an update, you can define the operation in a single flow by applying a condition in the update strategy.
  The following image shows an example of a condition applied to insert and update data in an Update Strategy expression:



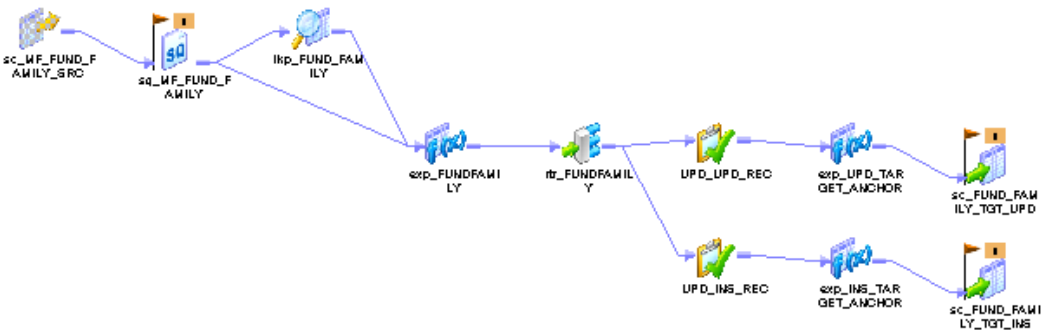
The following image shows a mapping configured with the Update Strategy expression:



- If you cannot avoid multiple inserts and updates on the same table, and if the data is of low volume, consider the following approach to avoid a serialization error:
  - Use a relational ODBC connection to insert or update data.
  - When the number of records that you want to update is high, do not use a relational connection as the updates are slow and occur at the row level.
  - You can break sessions which flag many records into two separate mappings or pipelines. When you configure separate pipelines, one for insert and one for update, you can use the Netezza Bulk writer in both the insert and the update flow to load the data.
- If you cannot avoid multiple inserts and updates on the same table and you have a large volume of data, consider the following approach to avoid a serialization error:
  - Check Ignore Key Constraints in the session properties when you want to insert data to a Netezza target. As Netezza does not enforce key constraints, the PowerCenter Integration Service performs additional processing when a session that writes to Netezza requires key constraints.
  - Configure the following properties for the Insert instance and Update instance:

Insert Instance	Update Instance
Writer Type = Bulk/PWX Inserts = Inserts Updates = None Deletes = False Ignore Key Constraint = True Duplicate Handling = FIRST or LAST Truncate Table = False	Writer Type = Bulk/PWX Inserts = None Updates = As Update Deletes = False Ignore Key Constraint = False Duplicate Handling = FIRST or LAST Truncate Table = False

The following image shows a mapping with the configured properties for the insert instance and update instance:



General Guidelines for Netezza Target Sessions

Consider the following general best practices when you configure the target properties for a session to write data to Netezza:

- The metadata of the target tables in the Netezza mappings must match the metadata in the Netezza database. If you make changes to the data in the database after you create the mappings, the session fails with the following error message:
  `[ERROR] The PowerCenter Integration Service fails the session, as Netezza might not be able to serialize execution of queries`
- If you set the truncate table option for a target in a session and if the Informatica ID does not have truncate privileges for the same table in the Netezza database, the session does not fail. You need to view the session log for the logged message.

Bulk Load

When the volume of data that you want to load into the target table is more than 10,000 rows, you can write records in bulk mode.

When you write bulk records to a Netezza target, specify the bad file name and path to capture rejected records, as the PowerCenter Integration Service does not create a bad file name, by default.

When you perform a bulk mode, you can use one of the following connection types:

ODBC Connection

You can enable the bulk load option with ODBC by setting the commit interval.

Configure the following parameters for ODBC to support transactions:

- Commit Interval. Enable this option to perform bulk operations when PowerExchange for Netezza is not available. Informatica recommends you to set a high value for this option for enhanced ODBC performance. You can use this option to avoid single row load operations while working with ODBC.
- Commit type. Use this option when you use the ODBC connection for bulk updates. Set this option in the target when you want to load data into Netezza.

PowerExchange for Netezza Connection

When you use the PowerExchange for Netezza connection, you can perform bulk load and unload to the target table using the external table. You must perform bulk load only when the data that you want to load to or unload from Netezza has more than 10,000 rows of data. Do not enable the commit interval and commit type options.

Single Data Row Inserts

When you perform a single data row insert, the PowerCenter Integration Service inserts a single row at a time into the target system. Single row inserts occur when you use the ODBC connection and do not set the commit interval. Use this mode to load or update less than 10,000 rows of data into Netezza or to check whether the extract, transfer, and load design functions correctly.

Single row update and insert performance is poor for the following reasons:

- Each update or insert requires that you compile an execution plan.
- Each update or insert requires that you lock the system catalog for a brief period.
- Each update or insert operation results in an external table creation, which contributes to catalog growth and catalog locking.
- Single row updates or inserts do not exploit the parallel processing power of PowerExchange for Netezza.
- Many single row runs can affect the performance due to the catalog impact.

The following table shows an example of a table definition and data for a single row insert:

Attribute	Type	Modifier
BRON	Character varying (3)	Not null
ID	Integer	Not null
GROUP_NR	Integer	Not null
TA_EXTRACT_DATETIME	Time stamp	-

To populate the entire table, a single load takes less than 10 seconds.

Multiple loads take 20 minutes with the following impact:

- Reduction in the load concurrency that results in using most of the 31 transaction slots, causing queuing.
- Locking of table catalog as a result of creating and dropping tables.
- Involves startup and closedown costs for each load, including factors such as logging.

Avoid single row loads either by using PowerExchange for Netezza or by setting the commit interval with ODBC. Using ODBC and setting the commit interval still results in multiple loads, but the performance impact decreases.

Concurrent Workflow

You can concurrently run more than one instance of a workflow.

Netezza allows only 31 concurrent read or write transactions to participate in series. If the system reaches this limit, and an implicit transaction occurs that attempts to modify data, the system puts this transaction in a queue. Such a transaction remains in the queue for 60 minutes, by default. After the timeout, the transaction fails and returns the following error message: `ERROR: Too many concurrent transactions.`

To change the default timeout setting, perform the following steps:

- To set the value for the current session, run the following command:

  ```
  SET serialization_queue_timeout = <number of minutes>
  ```

- To configure the global setting, set the variable `serialization_queue_timeout` in `postgresql.conf`.

The maximum number of concurrent workflows that you can run is a function of the number of target tables used in the mapping, the number of partitions, and the number of concurrent workflows run. If one parameter increases, appropriately adjust the other two to avoid serialization issues.

For more information about serialization_queue_timeout and begin_queue_if_full options, see the *Netezza System Administrator Guide*.

The key to determine how many workflows you can run concurrently depends on the following factors:

- The average number of targets associated with each workflow.
- The ability to manage the dependencies of the workflow order.

Informatica recommends that you create no more than four partitions or to run more than 20 concurrent workflows to avoid increasing the complexity of the design, with no substantial improvement in performance, and also to avoid encountering a serialization issue.

There is no direct option in PowerCenter to control the number of workflows submitted in parallel. You need to evaluate and understand the complexity of a workflow to determine whether to increase or reduce the number of workflows. To control the level of concurrency, use a job scheduler as a gatekeeper and PowerCenter workflow scheduler. If a workflow fails because of a serialization issue, the scheduler resubmits the workflow until it completes. Use the third-party schedulers that you can use with PowerCenter.

You can run concurrent workflows in the following scenarios:

Run without a Parameter File

The data that loads into the target table after running multiple concurrent workflows is based on the update strategy configured for the session. When you use the insert operation, the same copy of the source data loads to the target table. The PowerCenter Integration Service performs this operation for all the workflows that you run. Informatica recommends not to use concurrent workflows. Do not use the update or delete options because Netezza does not allow parallel update or delete operations on the same table.

Run with a Parameter File

You can use a parameter file to pass certain settings for the workflows without the requirement to edit the actual workflow. You can configure each concurrent workflow to use a separate parameter file. You can use the parameters in the file to specify the external table settings used for bulk loads or updates. Informatica recommends that you should not use different external table settings with the same set of source data because incorrect data might be inserted into the target.

For example, when you change the null value setting or the data delimiter setting, the same data passes through external tables and result in loading incorrect data. Therefore, the parameters remain the same for all the concurrent workflows, but the connection objects used by the workflows must be different. You can use this setup to run the same workflow on different databases on the same or different Netezza systems.

When you use the ODBC connection, you can also pass the commit interval value by using a parameter file.

Setting a Parameter File for a Workflow

You want to create two parameter files in which you want to configure all the parameters. Before running the workflow, you must specify the parameter file.

The following image shows the configured parameter file `param_reader.txt` in the session properties:

For more information about the parameters and syntax for creating a parameter file, see the *Informatica PowerCenter Advanced Workflow Guide*.

Session Property Recommendations for ODBC Settings

You can use either PowerExchange for Netezza or the default ODBC connection to connect to Netezza. With PowerExchange for Netezza, bulk load is efficient and performance is faster compared to the ODBC connection.

If you want to configure pushdown optimization and lookup on Netezza target table, use the ODBC connection. ODBC connection does not support duplicate row handling. With ODBC connection, bulk load is possible only to a certain extent depending on the configured commit interval value.

Consider the following recommendations when you use the ODBC connection:

Load

Informatica recommends to specify the Pipe Directory Path option in the session to the local file system for better performance. Socket buffer size is the amount of data read or sent at a time. Default bucket buffer size is 8 K, which increases or decreases the load performance. If sufficient network bandwidth is available, set the socket buffer size to a larger value to improve the performance.

Enable load continuation only if it is required to allocate extra memory because the operation causes an overhead. Load continuation has bigger performance impact. Intermediate checkpoints for each load slows the load even if there are no SPU failures. Turning off the load continuation implies that if a SPU fails while the load is in progress, the load does not continue and exits.

Unload

Host side unload is faster than the remote unload. Host side is the local file system on the Netezza host. Informatica recommends to specify the Pipe Directory Path option in the session to the local file system for better performance.

Delimiter

Consider the following recommendations for delimiters:

- A delimiter must be a single character.
- A delimiter must be different than the data in the field, especially char or varchar data.
- The date and time delimiters must be different from the field delimiter.
- The default delimiter is tab (‘\t’). When ‘\t’ is the delimiter for a Netezza source in the session properties, the PowerCenter Integration Service truncates the target data. You must set the delimiter to a different value.
- Set a delimiter to a value other than 7 bit ASCII. In this case, specify the decimal or hex value of the delimiter using the delim option.

NullValue

Consider the following recommendations for null values:

- The null value can be an empty string or a value in the range of a-z or A-Z. Default value is NULL.
- The null value must be a single character for PowerExchange for Netezza.
- When you need to extract non-null values from a Netezza source, the PowerCenter Integration Service also extracts empty strings. These values might appear as null values in the target.

EscapeCharacter

Consider the following recommendations for escape characters:

- With –escapeChar, you can specify only ‘\’. The default is NO ESCAPE char.
- Use escape processing if data or field values contain field-delimiter or new-line or zero-byte (\0), irrespective of any other settings.
- If you specify EscapeChar, the data values with “\” are escaped. Additionally, depending on the ControlChar flag and CrInstring flags used, all characters between 0-31 must be escaped if they are present in data-fields.
- Depending on the NullValue setting, if the nullvalue string itself is the data value and is not to be treated as NULL for a particular instance, it must be escaped.

ErrorLogDirectoryName

Default is “/tmp” for external table queries on UNIX platforms. The PowerCenter Integration Service creates a bad file in the error log directory if the data is not valid. For multiple partitions, Informatica recommends to specify a unique ErrorLogDirectoryName for each partition to preserve information about the bad records, if any.

Truncate Target Table Option

Informatica recommends to truncate instead of dropping and recreating the table to avoid catalog growth.

When you load data directly from the source table to the production table, Informatica recommends you to avoid data loss.

For a production system, create a copy of the source table before you begin loading the data.

For example, use the following syntax to make a copy:

```
CREATE TABLE loan_backup AS SELECT * FROM loan;
```

You can run this statement as a part of the pre-SQL command.

Control Character and CRINSTRING

Use the Control Character and CRINSTRING flags to parse the data that you want to load. Setting these flags on or off affects the performance.

Quoted Value

Quoted values require additional parsing. Therefore, if the data is not in the quoted value format, do not set this option for performance benefits.

Ignore Zero Value

You cannot use unescaped zero-byte. If you want to include zero-byte as the part of the valid data value, you must escape and set the IgnoreZero flag to False. If you want to ignore zero-byte for all data-values, do not escape for the zero-byte and set IgnoreZero flag to True. By default, the IgnoreZero flag is set to False.

Ignore Key Constraint

The performance of the connector improves when you enable the Ignore Key Constraint option. If both source and destination are Netezza systems, a better option is to enable the Distinct flag while extracting the data from Netezza, and then enable the Ignore Key Constraint flag. This configuration manages duplicates at the source, thus improving the performance.

Connection Attribute Information

By default, Netezza listens on port 5480. When the PowerCenter Integration Service runs in Unicode mode, it encodes Netezza data of the Nchar (m) and NVarchar (m) data types in UTF-8. The PowerCenter Integration Service encodes Netezza data of type Varchar and Char in Latin-9. If the data contains extended ASCII characters or UTF-8 characters, run the PowerCenter Integration Service in Unicode mode.

Pre-SQL and Post-SQL Commands

You can use pre-SQL and post-SQL commands to perform specific operations before and after the actual workflow execution.

You can use the commands to optimize performance or perform database functions outside an Informatica mapping. Ensure that you run the queries because part of pre-SQL and post-SQL commands are not performance intensive that you can use mostly to set up the environment or for a clean-up.

You can use pre-SQL and post-SQL commands for the following scenarios:

Pre-SQL Commands

Disable Mviews before you insert, update, or delete data from the associated target table to optimize performance.

Post-SQL Commands

Re-create the Mviews or run update statistics on a table altered due to Informatica mapping.

Stored Procedure Calls

You can run a Netezza stored procedure by calling the stored procedure from a pre-session command, post-session command, or from a command task. Ensure that you do not call a Netezza stored procedure from a Stored Procedure transformation in PowerCenter.

Avoiding Common Errors with PowerExchange for Netezza

When you read or write data to Netezza, consider the following configurations to avoid common issues:

- Alternatives to partitioning
- Serialization errors
- Serialize transaction errors
- Unavailability of locks
- Buffer sizes

Alternatives to Partitioning

When loading or unloading data to or from Netezza, you can configure the mapping without using partitioning by any of the following methods:

Run multiple sessions of the same mapping

You can configure multiple sessions of the same mapping and run them concurrently with the sessions working on mutually exclusive records, called as classic divide and conquer.

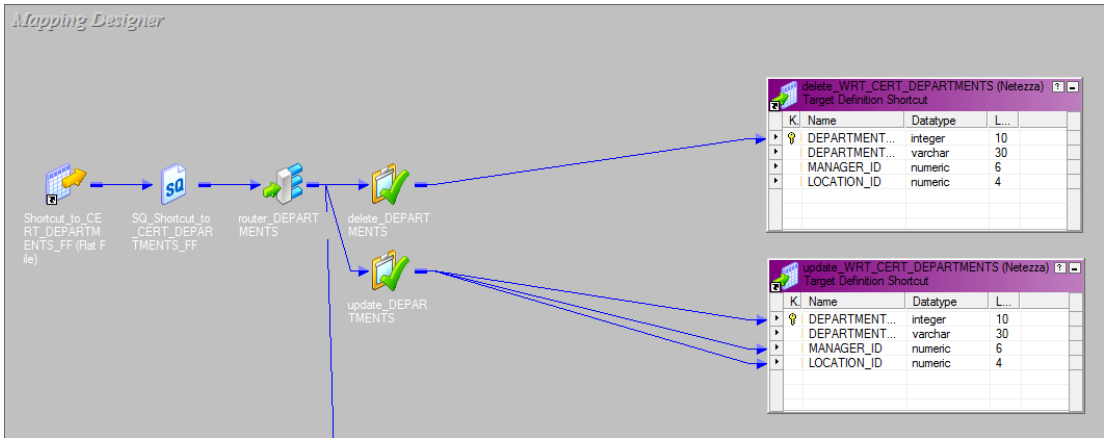
Add more CPU resources

Improve transformation speeds by adding more CPU resources. You can add CPUs in a single thread mode and without the parallel option. Performance increases with the complexity of the transformation.

Serialization Errors

Serialization issues occur due to serialization errors in the Netezza target when you run multiple delete, update, and insert queries with where clauses on the same table and do not commit or roll back.

The following image shows an example mapping:



You can view two instances of the same Netezza table as targets in the mapping. The first pipeline deletes rows from the Netezza target, and the second pipeline inserts the rows into the same Netezza target. The session update or insert rows to the Netezza target.

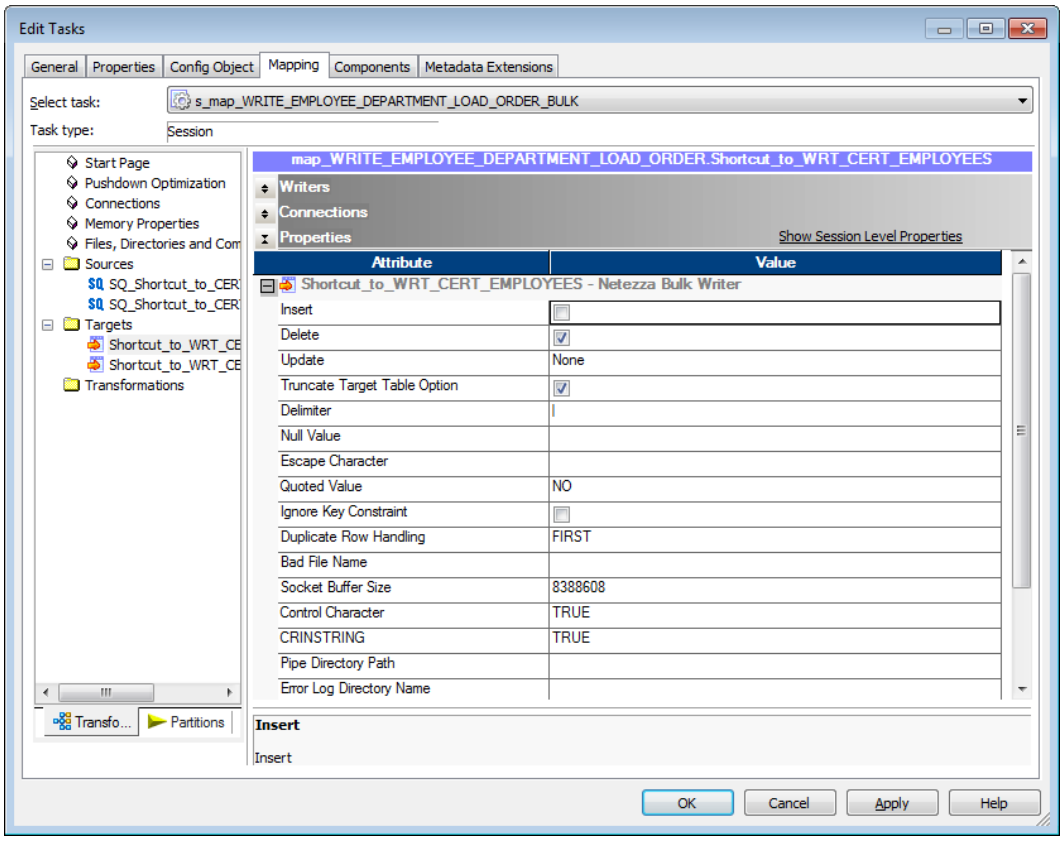
Consider the following requirements when using multiple instances of the same target:

- The pipeline that uses the Insert configuration must have the Ignore Key Constraint flag enabled.
- An update strategy with the combination of delete and update or insert, with duplicate row handling and update fails with the serialization error.

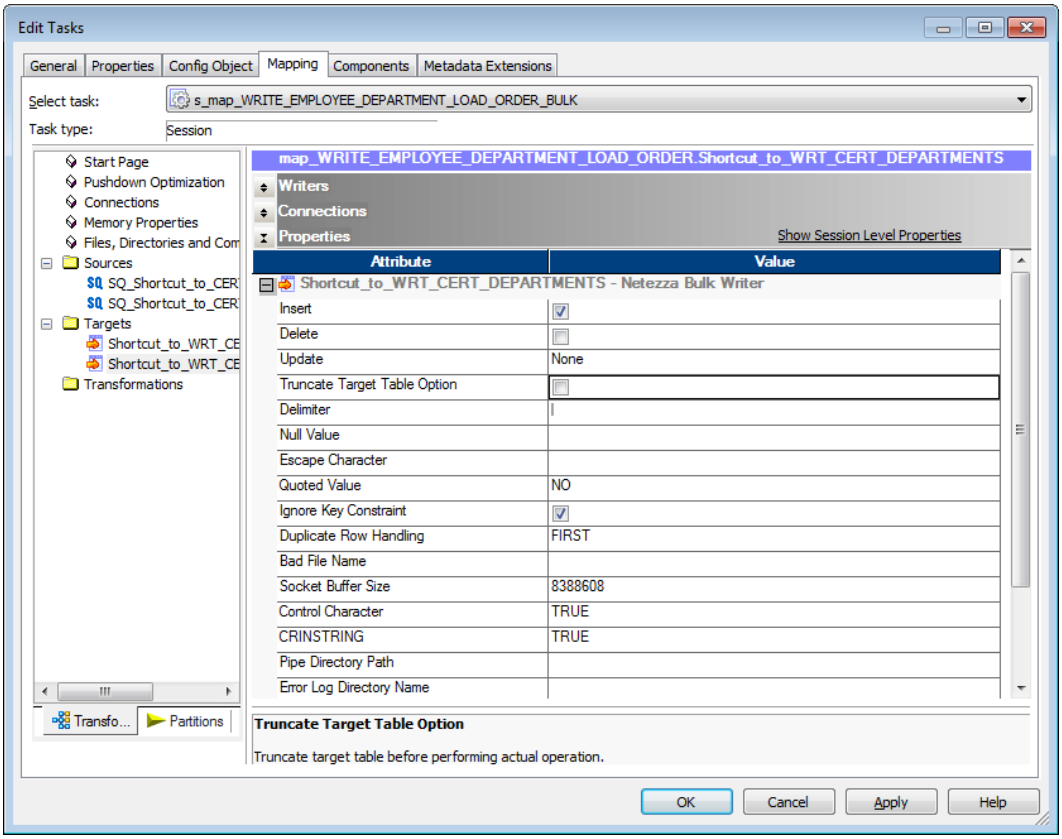
To resolve the issue, configure the following settings in the session:

1. Select **Treat source rows** as data driven.
2. In the first target instance, select only the **Delete** check box.

The following image shows the settings that you can configure:



3. In the second target instance, select the **Insert** and **Ignore Key Constraint** check boxes.
   The following image shows the configured settings:



Serializable Transaction Isolation

The ANSI/ISO SQL standard defines the following levels of transaction isolation:

- Uncommitted read
- Committed read
- Repeatable read
- Serializable

The Netezza system implements serializable transaction isolation, which provides the highest level of consistency. If two concurrent transactions attempt to modify the same data, the system rolls back the latest transaction. This form of optimistic concurrency control is suitable for low-conflict environments such as data warehouses.

Scenarios 1 and 2 are examples that might result in `Could not serialize - transaction aborted` errors.

The following table structure is used for the scenarios:

```
Table Name: Student
Database Name: Dev
User: Admin
```

The following table shows the data loaded into the Student table:

Roll_No	Name	Course
101	Martin	BS
202	Bob	MS
303	Ryan	PhD

Scenario 1

1. Start a nzsql session and enter the following queries:

```
BEGIN;
SELECT * FROM Student;
INSERT INTO Student VALUES (404, 'Jon', 'BS');
```

2. Start another session and enter the following queries:

```
BEGIN;
INSERT INTO Student VALUES (505, 'Smith', 'BS');
SELECT * FROM Student;
```

The second session results in the following error: `ERROR: DEV.ADMIN.STUDENT : Could not serialize - transaction aborted`

Scenario 2

1. Start a nzsql session and enter the following queries:

```
BEGIN;
UPDATE Student SET Name='Smith' WHERE Roll=101;
Start another nzsql session and enter the following queries:
BEGIN;
UPDATE Student SET Name='Jon' WHERE Roll=202;
```

The second session results in the following error: `ERROR: DEV.ADMIN.STUDENT : Could not serialize - transaction aborted`

When there is a conflict among concurrent transactions, Netezza reports an error. To avoid this situation, verify that there are no cycles in concurrent transactions.

Unavailability of Locks on Netezza Tables

One of the reasons a workflow waits for the other concurrent workflow to complete is because of the unavailability of a lock on the table. Verify by using the Netezza show locks command or by verifying the contents of the _t_pg_locks table.

Locks are used to ensure that only one user can modify a record at a time and that there are no reads that are not valid.

You can use the following types of locks in Netezza:

Access Share Lock

Performs read operations.

Row Exclusive Lock

Performs update operations.

Access Exclusive Lock

Performs DDL operations as a part of pre-SQL and post-SQL commands.

Netezza maintains the lock information in a system table.

To verify the details of locks acquired by different processes at any point of time, perform the following steps:

Start an nzsql session and run the following command:

```
Show locks;
```

or

```
SELECT * FROM _t_pg_locks;
```

Consider the following example where a user query is unable to proceed due to a lock issue. There are two users, A and B. User A connects to the database MyDatabase and runs the following query:

```
BEGIN:
INSERT INTO Student VALUES (505, 'Jon', 'Phd');
```

User B connects to the same database MyDatabase and submits the following query:

```
TRUNCATE TABLE Student;
```

The query submitted by User B will not proceed until User A enters the command `ROLLBACK` or `COMMIT`.

User B as an admin user can run the `show locks;` command to confirm the details of the locks acquired by different sessions. Alternatively, User B can check the contents of the _t_pg_locks if SELECT permission on the t_pg_locks is granted to User B.

The following table shows the output of the show locks command:

SESSION ID	DATA BASEID	RELID	USER NAME	PROCESS ID	CLIENT IP	LOCK STATE	LOCK MODE REQUESTTIME	REQUEST TIME	GRANT TIME	COMMAND
16084	200075	200534	A	29267	10.77. 1.140	HOLD	AccessShare Lock	2009-11-11 08:27:12	2009-11-11 08:27:12	INSERT INTO Student VALUES(505, 'Jon', 'Phd');
16084	200075	200534	A	29267	10.77. 1.140	HOLD	RowExclusive Lock	2009-11-11 08:27:10	2009-11-11 08:27:10	INSERT INTO Student VALUES(505, 'Jon','Phd');
16085	200075	200534	B	29278	10.77. 1.140	WAIT	AccessExclusive Lock	2009-11-11 08:27:25	-	TRUNCATE TABLE Student;

The following variables describe the columns in the table:

- SESSIONID: The user session ID that holds or waits for the lock.
- DATABASEID: The database ID to which the session is connected.
- RELID: The relation ID for which the lock is requested.
- USERNAME: The user name associated with the session ID.
- PROCESSID: The process ID associated with the session.
- CLIENTIP: The IP address of the client machine.
- LOCKSTATE: The status of the lock, hold or wait.
- LOCKMODE: The lock mode, whether acquired or requested.
- REQUESTTIME: The time when the lock was requested.
- GRANTTIME: The time when the lock was granted.

- COMMAND: The user command to request the lock.

The table shows that the query submitted by User B with session ID 16085 waits to acquire an AccessExclusiveLock on relation ID 200534, Student Table.

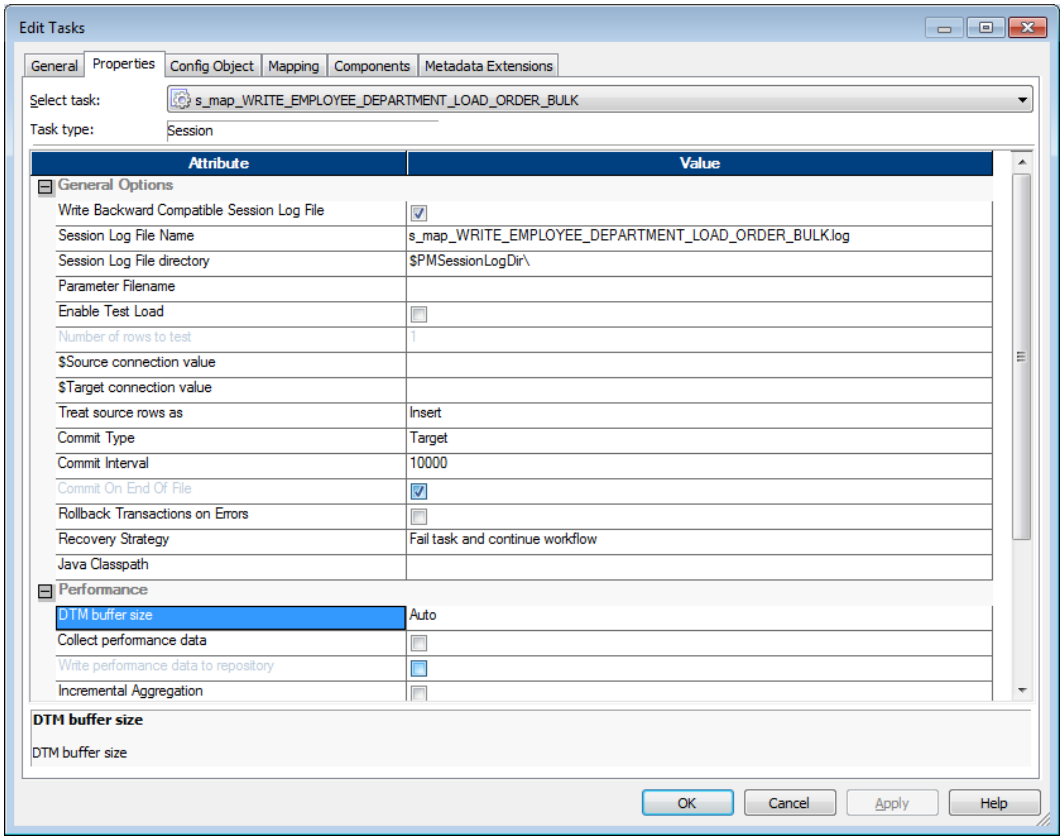
Buffer Size

Consider the following recommendations when you specify the buffer size:

DTM Buffer Size

You can increase or decrease the value of the DTM buffer size to specify the amount of memory that the PowerCenter Integration Service uses as DTM buffer. The exact size of the buffer depends on multiple parameters such as load size, source, and destination. When you set the DTM buffer size as Auto, the maximum DTM buffer size is 512 MB, or 5% of the total memory.

The following image shows the buffer size settings:



Line Sequential Buffer Length

You can improve the session performance by setting the number of bytes that the PowerCenter Integration Service reads for each line. The exact size of the buffer depends on multiple parameters such as load size, source, and destination.

Default Buffer Block Size

You can increase or decrease the number of available memory blocks that the PowerCenter Integration Service uses to hold the source and target data in a session. The exact size of the buffer depends on PowerCenter parameters such as load size, source, and destination.

Authors

Melufa M.M

Dimple Rai

Acknowledgements

The authors would like to acknowledge Muralidharareddy Gali for his contribution to this article.